Pseudo-code for the 'RPi_selectorPI_operative.py' file

Davide Carecci; reserved for A2A S.p.A.

October 29, 2025

In the following, ***bold*** refers to vectorization in the variables spaces, whereas the time vectorization is implicit. When time indexing is relevant, it is explicitly reported as subscipt.

Nomenclature

$(x_{C,gb}/x_{M,gb})_{ub,high}$ Higher hysteresis' activation threshold for $x_{C,gb}/x_{M,gb}$ upper bound

$(x_{C,gb}/x_{M,gb})_{ub,low}$ Lower hysteresis' activation threshold for $x_{C,gb}/x_{M,gb}$ upper bound

χ_{danger} Boolean variable that points out the safety condition of the working point

$\tilde{\mathbf{y}}$ *Online* (automatic, high-frequency) measured data, already pre-processed

$\mathbf{e_y}$ Output tracking errors

$\mathbf{T}$ Vector of time instants to perform integration over

$\mathbf{y_{ref}}$ Output setpoints set by agri-AcoDM

$q_{\tilde{M},gb}$ Measured methane flow rate (in L h^{-1})

$q_{\tilde{TOT}}$ Measured total biogas flow rate (in L h^{-1})

$x_{C,gb}\tilde{/}x_{M,gb}$ Measured biogas composition ratio (dimensionless)

$x_{\tilde{C},gb}$ Measured CO_2 volume fraction in biogas (dimensionless)

$x_{\tilde{M},gb}$ Measured CO_2 volume fraction in biogas (dimensionless)

I Accumulated integral term of the PI controller

k Index of the control run

k_P Proportional gain of the PI controller

T_c Control interval

T_I Integral time constant of the PI controller (in seconds)

u Feedback correction/deviation of the control action computed by the controller

u_0	Equilibrium input/feedforward control action of the controller
u_I	Integral component of the control action of the PI controller
u_P	Proportional component of the control action of the PI controller
u_{actual}	Actual actuator command
u_{max}	Upper bound saturation of u
u_{min}	Lower bound saturation of u
var^+	Updated value of the var variable
var^-	"Preliminary" computation for the value of the var variable
$var^{(i)}$	ith entry of the **var** vector
var_k	Value at the k^{th} control run of the var variable. Also as $var_{sub}\mid_k$ for the var_{sub} variable

Pseudo-code

Algorithm 1 main.py

for k in range(0, T_c, end of experiment) **do**
  Initialize Logger
  Round the timestamp correspondent to k to HH:00:00 (*now*)
  Read the CSV file containing $\tilde{\mathbf{y}} = [q_{\tilde{TOT}}, x_{\tilde{M},gb}, x_{\tilde{C},gb}]$
  Filter $\tilde{\mathbf{y}}$ between *start* to *end* timestamps (*end* = *now*)
  Compute $q_{\tilde{M},gb}$ and $x_{C,gb}/\tilde{x}_{M,gb}$. Extract $q_{\tilde{M},gb}|_k$ and $(x_{C,gb}/\tilde{x}_{M,gb})_k$
  Read the TXT file containing $\mathbf{y_{ref}} = [q_{M,gb,ref}, (x_{C,gb}/x_{M,gb})_{ref}]$ (setpoints set by agri-AcoDM), $(x_{C,gb}/x_{M,gb})_{ub,low}$ and $(x_{C,gb}/x_{M,gb})_{ub,high}$. Extract $q_{M,gb,ref}|_k$ and $(x_{C,gb}/x_{M,gb})_{ref}|_k$
  Save $\tilde{\mathbf{y}}|_k$ and $\mathbf{y_{ref}}|_k$ to CSV file
  Compute $\mathbf{e_y}|_k = [q_{\tilde{M},gb}|_k - q_{M,gb,ref}|_k, (x_{C,gb}/\tilde{x}_{M,gb})_k - (x_{C,gb}/x_{M,gb})_{ref}|_k]$
  Instantiate PIController `class` of the SelectorPI_controller.py file as *header* ($\rightarrow q_{\tilde{M},gb}$ PI controller) and *follower* ($\rightarrow x_{C,gb}/\tilde{x}_{M,gb}$ PI controller)
  procedure object.load_state(date, filename) **for** object $\in$ [*header*,*follower*]
    return I_{k-1}, $\chi_{\text{danger}}|_{k-1}$
  end procedure
  Instantiate HysteresisComparator `class` of the SelectorPI_controller.py file
  procedure HysteresisComparator.update($(x_{C,gb}/\tilde{x}_{M,gb})_k$, $\chi_{\text{danger}}|_{k-1}$)
    $\chi_{\text{danger}}|_k$ = **not** $\chi_{\text{danger}}|_{k-1}$ **and** $(x_{C,gb}/\tilde{x}_{M,gb})_k > (x_{C,gb}/x_{M,gb})_{ub,high}$ **or** $\chi_{\text{danger}}|_{k-1}$ **and** $(x_{C,gb}/\tilde{x}_{M,gb})_k \geq (x_{C,gb}/x_{M,gb})_{ub,low}$
    return $\chi_{\text{danger}}|_k$
  end procedure
  if $\chi_{\text{danger}}|_k \neq \chi_{\text{danger}}|_{k-1}$ **and not** $\chi_{\text{danger}}|_{k-1}$ **then**
    Compute I^+_{k-1} = *header.u* - *follower.*$k_P \cdot e_y^{(0)}|_k \cdot \frac{follower.T_I}{follower.k_P}$
    procedure *follower*.reset_state(I^+_{k-1})
      Override I_{k-1} with I^+_{k-1}
    end procedure
  else if $\chi_{\text{danger}}|_k \neq \chi_{\text{danger}}|_{k-1}$ **and** $\chi_{\text{danger}}|_{k-1}$ **then**
    Compute I^+_{k-1} = *follower.u* - *header.*$k_P \cdot e_y^{(1)}|_k \cdot \frac{header.T_I}{header.k_P}$
    procedure *header*.reset_state(I^+_{k-1})
      Override I_{k-1} with I^+_{k-1}
    end procedure
  else break
  end if
  procedure object.compute(error$|_k$) **for** object $\in$ [*header*, *follower*], error $\in$ [$e_y^{(0)}$, $e_y^{(1)}$]
    $u_P|_k = k_P \cdot$ error
    $I_k = I_{k-1} + T_c \cdot$ error
    $u_I|_k = I_k \cdot \frac{k_P}{T_I}$
    $u_k^- = u_P|_k + u_I|_k$
    Apply saturation: u_k = **if** $u_k^- \leq u_{min}$ **then** u_{min} **else if** $u_k^- \geq u_{max}$ **then** u_{max} **else** u_k^-
    Apply anti-windup: I_k^+ = **if** $u_k^- \leq u_{min}$ **or** $u_k^- \geq u_{max}$ **then** $(u_k - u_P) \cdot \frac{T_I}{k_P}$ **else** I_k
  end procedure

39: **procedure** OBJECT.SAVE_STATE(date, filename) **for** OBJECT $\in$ [*header*,*follower*]
40: Save to CSV file I_k^+, $\chi_{\text{danger}}|_k$
41: **end procedure**
42: Select $u_k =$ **if** $\chi_{\text{danger}}|_k$ **then** $follower.u_k$ **else** $header.u_k$
43: Compute $u_{actual}|_k = u_0 + u_k$
44: Convert $u_{actual}|_k$ to *on/off* times for the PWM signal that controls the peristaltic pump
45: Save results to CSV file. Plot and save to PNG files
46: **end for**

Algorithm 1 main.py (simplified)

1: Initialize Logger
2: Round to HH:00:00 the current control run timestamp
3: Read from CSV files *all* past *online* measurement data (alredy pre-processed with outlier detection and moving average for the total biogas flow rate). Cut between start/end timestamps.
4: Compute methane flow rate and biogas compositon ratio from data. Extract the values corresponding to the current control run timestamp
5: Read from TXT file the outputs' setpoints and the activation thresholds of the hysteresis logic
6: Save measured and setpoint values at the current control run timestamp to CSV for checking and plotting purposes
7: Compute the feedback errors
8: Instantiate the PICONTROLLER `class` of the SELECTORPI_CONTROLLER.PY file as *header* ($\rightarrow$ methane flow rate PI controller) and *follower* ($\rightarrow$ biogas ratio PI controller)
9: Read from CSV file: (i) the values of the PI integrators' states and (ii) the boolean condition that indicates too high/unsafe values of the biogas ratio, at the previous control run timestamp
10: Instantiate the HYSTERESISCOMPARATOR `class` of the SELECTORPI_CONTROLLER.PY file
11: **if** Biogas ratio > 'safety' threshold **then**
12: Set boolean condition to true i.e. activate *follower*
13: **end if**
14: **if** *follower* was activated now **then**
15: Update the integral term of the *follower* to be consistent with the current control action
16: **else if** *header* was activated now **then**
17: Update the integral term of the *header* to be consistent with the current control action
18: **end if**
19: For both controllers compute the new control action's correction/deviation as a standard PI with saturation and anti-windup
20: Save results to CSV files
21: Select
22: Add the equilibrium/feedforward input to the correction/deviation computed by the controller to obtain the actual actuator command.
23: Convert the value of the actual actuator command to *on/off* times for the PWM signal that controls the peristaltic pump
24: Plot results and save to PNG files

Actually, the 'main.py' function is called at each control step k by the 'cronjob.py' file, so that in practice the **for** loop is not present in the 'RPi_selectorPI_operative.py' file. Note: at initialization,

$I_0 = 0$ and $\chi_{danger}|_0$ is false.